Independent Program Verification Outperforms Translation in Diagnosing Reasoning Errors in Small Language Models: A Causal and Metacognitive Framework

Student Research Group (NCKH)
Department of Computer Science, University of Science, VNU-HCM
student-research@vnu-science.edu.vn

Academic Year 2026–2027

Abstract

Chain-of-thought (CoT) prompting frequently suffers from post-hoc rationalization, wherein intermediate verbalized rationales fail to causally govern model answers or merely mimic flawed heuristic assumptions. In this paper, we investigate whether deterministic programmatic verifiers and structured reasoning representations can reliably diagnose reasoning fallacies and execution slips in fine-tuned 8-billion parameter models (Qwen3-8B 4-bit with LoRA) without human intervention. We establish a pre-registered experimental evaluation across 500 challenging problems from the MATH dataset (Levels 3–5) and GSM8K. On GSM8K, structured `Step/Calc` fine-tuning achieves 95/100–97/100 zero-prefill format validity (passing Gate 0), high causal faithfulness under intermediate arithmetic perturbations (89.7%–93.8% causal following, with only 2.8%–3.1% reverting to unperturbed beliefs), and recovers 68.4%–78.9% (13/19 to 15/19) of legacy failure targets via Step 1 semantic enrichment (restoring 87/100 zero-prefill accuracy under `enriched_cf`, McNemar $p = 0.1094$ vs. `base_prefill`). However, scaling flat structured traces to symbolic mathematics reveals a fundamental structural bottleneck: internal equational verification is insufficient, catching only 2.4% (2/82) of erroneous solutions on MATH Levels 3–5 (items 101–300 under `base_showcalc`, false alarm rate 0/118), and although non-truncated LoRA traces achieve 72.7% (40/55) accuracy, paragraph-collapsed proofs induce greedy repetition loops in 75.6% (34/45) of truncated sequences. Under rigorous double-blind adjudication ($\kappa = 0.947$), our pre-registered confirmation study on MATH (items 301–500) proves the **Circular Confirmation Trap**: **independent programmatic generation catches 5× more reasoning errors than literal thought-trace translation** (10/24 [41.7%] vs. 2/24 [8.3%], exact two-sided McNemar $p = 0.0215 < 0.05$), while translation achieves parity on pure execution arithmetic (69.2% vs. 61.5%). To resolve the flat-trace repetition versus monolithic-interpreter crash trade-off on symbolic mathematics, we outline an encapsulated Directed Acyclic Graph (DAG) and selective tool-delegation architecture (MSRA) as future work.

1 Introduction

Modern autoregressive large language models demonstrate competitive performance when generating natural language rationales step by step [Wei et al., 2022, Cobbe et al., 2021]. Despite these empirical gains, extensive evidence reveals that verbalized rationales often function as post-hoc justifications rather than faithful computational engines [Lanham et al., 2023, Turpin et al., 2024]. In complex mathematical problem solving, reasoning errors fall into two fundamentally distinct categories:

1. **Execution Slips:** Arithmetic miscalculations where the underlying logical formula or conceptual relationship is correct, but the discrete evaluation is erroneous (e.g., $14 \times 7 = 96$).

2. **Reasoning Fallacies:** Conceptual formulation breakdowns where the model asserts an invalid mathematical premise, ignores physical constraints, or constructs a fundamentally incorrect equation (e.g., equating perimeter directly to area).

To mitigate arithmetic failure modes, prior works such as Program-Aided Language Models (PAL) [Gao et al., 2023] and Program of Thoughts (PoT) [Chen et al., 2023] outsource arithmetic evaluation to an external Python interpreter. When applying programmatic verification to monitor autoregressive generation, two divergent paradigms emerge:

- **Translated Trace Verification ($\mathcal{T}$):** The model translates its natural language thought trace into Python/SymPy code, which is then executed to verify whether the textually claimed answer matches the executed value.
- **Independent Program Verification ($\mathcal{I}$):** An external solver script is generated directly and independently from the problem text Q, bypassing the internal textual rationale.

A central dilemma arises: *Does translating the model's textual thinking into code enhance diagnostic sensitivity, or does it merely automate the transcription of the model's hallucinations?*

In this work, we address these fundamental questions through a multi-phase pre-registered study on fine-tuned Qwen3-8B [Yang et al., 2024] across multi-step arithmetic (GSM8K) and challenging symbolic mathematics (MATH Levels 3–5) [Hendrycks et al., 2021]. We establish three core findings:

1. **Structured Traces & Causal Faithfulness on Arithmetic:** Fine-tuning with explicit `Step`/`Calc` delimiters enforces high format compliance (95/100–97/100, passing Gate 0) and high causal faithfulness under intermediate arithmetic perturbations (89.7%–93.8% following rate, with only 2.8%–3.1% reverting to prior unperturbed answers). Combining Step 1 semantic enrichment with counterfactual augmentation (`enriched_cf`) recovers 78.9% (15/19) of legacy failure targets zero-prefill (matching the 15/19 ceiling of runtime prefix prompting) and achieves 87/100 overall accuracy (McNemar $p = 0.1094$ vs. `base_prefill`).
2. **The Flat-Trace Bottleneck on Symbolic Mathematics:** When scaling flat `Step`/`Calc` traces to MATH Levels 3–5, simple internal equational verification fails, catching only 2.4% (2/82) of erroneous solutions. Moreover, while non-truncated LoRA completions achieve 72.7% (40/55) accuracy, paragraph-collapsed proofs trigger severe greedy repetition loops in 75.6% (34/45) of truncated generations, highlighting the limitation of unconstrained flat autoregression on symbolic proofs.
3. **The Circular Confirmation Trap:** We prove mathematically and confirm empirically under double-blind adjudication ($\kappa = 0.947$) that translated verification suffers from circular confirmation: high translation fidelity causes code translation to faithfully duplicate reasoning fallacies with high probability, rendering it blind to conceptual errors (catching only 8.3% of reasoning bugs). In contrast, independent programmatic verification exposes reasoning fallacies with 5× higher recall (41.7%, exact two-sided McNemar $p = 0.0215$), while matching translation on execution slips (61.5% vs. 69.2%).

Finally, to address the dilemma where flat autoregressive traces collapse into repetition loops while monolithic programmatic solvers fail execution on complex symbolic queries (27/100 crashes in PAL), we outline an encapsulated Directed Acyclic Graph (DAG) reasoning agent with reference-free preference alignment (MSRA) as a principled direction for future work.

2 Related Work & Theoretical Foundations

2.1 Rationale Faithfulness and Post-Hoc Confabulation

The causal relationship between language model rationales and downstream answers has been extensively questioned. Lanham et al. [2023] demonstrated via feature attribution and prompt

perturbation that models frequently generate identical answers even when extensive portions of their reasoning traces are truncated or modified. Turpin et al. [2024] revealed that injecting biased features into the context causes models to systematically alter their answers while systematically omitting the biasing influence from their stated rationales. Lyu et al. [2023] proposed restructuring rationales into deterministic dependency chains, though primarily evaluated on synthetic tasks without assessing error diagnosis.

2.2 Programmatic Reasoning and Process Verifiers

Outsourcing intermediate computation to deterministic interpreters represents a prominent paradigm for eliminating arithmetic drift. PAL [Gao et al., 2023] and PoT [Chen et al., 2023] demonstrated that generating Python scripts outperforms standard CoT on arithmetic benchmarks. In parallel, Process Reward Models (PRMs) [Lightman et al., 2023, Uesato et al., 2022] train learned neural verifiers to assign step-by-step correctness scores. However, PRMs require millions of step-level human or synthetic annotations and demand substantial inference-time compute. In contrast, our study focuses on lightweight, deterministic programmatic checking that executes in milliseconds without auxiliary neural verifiers.

2.3 Theoretical Analysis: The Circular Confirmation Trap

We provide a formal mathematical model explaining why translated verification fails to catch reasoning bugs.

Definition 1 (Reasoning and Program Variables)**.** *Let $Q \in \mathcal{X}$ denote the input query. Let $T \in \mathcal{T}$ denote the autoregressive natural language thought trace generated by policy $\pi_\theta(T \mid Q)$, and $A \in \mathcal{Y}$ denote the final answer. Let $C_T \sim \pi_{trans}(C \mid Q, T)$ denote the translated program, and $C_I \sim \pi_{indep}(C \mid Q)$ denote the independently generated program. Let $y^* \in \mathcal{Y}$ denote the ground-truth solution.*

Definition 2 (Error Decomposition)**.** *We partition failures $A \neq y^*$ into two disjoint error sets:*

1. *An **Execution Slip** (E_{exec}): The mathematical relations in T are conceptually valid, but manual computation produces an arithmetic error.*
2. *A **Reasoning Fallacy** (E_{reason}): The mathematical relations asserted in T are conceptually false, contradictory, or structurally invalid.*

Theorem 1 (Circular Confirmation Bound in Translation)**.** *Assume policy π_{trans} has semantic fidelity $\phi \in (0, 1)$ such that for any premise $p \in T$, the probability that C_T encodes p is at least ϕ. Then for any reasoning fallacy $E_{reason} \in T$ governed by flawed premise p_{flaw}, the probability that translated execution C_T detects the fallacy satisfies:*

$$\mathbb{P}(Detect(C_T) \mid E_{reason}) \leq 1 - \phi \tag{1}$$

Proof. Let $\text{Eval}(C)$ denote the executed output of program C. By definition, detection occurs when $\text{Eval}(C_T) \neq A$. Under error condition E_{reason}, the model asserts answer $A = f(p_{\text{flaw}})$. If the translator faithfully transcribes p_{flaw} with probability ϕ, the program executes $\text{Eval}(C_T) = f(p_{\text{flaw}}) = A$. Hence, $\mathbb{P}(\text{Eval}(C_T) = A \mid E_{\text{reason}}) \geq \phi$, which implies $\mathbb{P}(\text{Detect}(C_T) \mid E_{\text{reason}}) \leq 1 - \phi$. □

Corollary 2 (Fidelity-Detection Trade-off)**.** *As the language model's code translation capability improves ($\phi \to 1$), its capacity to detect reasoning errors via translated trace checking approaches zero ($\mathbb{P}(Detect(C_T) \mid E_{reason}) \to 0$).*

In contrast, independent programmatic generation C_I is sampled from $\pi_{\text{indep}}(C \mid Q)$, conditionally independent of the natural language trace T:

$$\mathbb{P}(\text{Detect}(C_I) \mid E_{\text{reason}}) = \mathbb{P}(\text{Eval}(C_I) \neq A \mid E_{\text{reason}}) \tag{2}$$

Because the generation of C_I requires constructing an algorithmic solver from scratch, it is not bound to repeat the hallucinated intermediate premise p_{flaw}, achieving strictly higher recall on reasoning bugs.

3 Empirical Evaluation & Pre-Registered Results

3.1 Experimental Setup & Pre-Registration Protocol

All experiments are conducted on Qwen3-8B 4-bit fine-tuned with LoRA ($r = 8$, scale 20.0, dropout 0.0, 8 target layers, learning rate 5×10^{-5}, training `--max-seq-length 512` on 16 GB Apple M1) using MLX.

- **MATH Dataset Partitions:** 1,301 deterministic checkable problems from Levels 3–5 partitioned via SHA-256 hash ordering into 725 train, 76 valid, and 500 dev problems.
- **Strict Disjointness:** Official test sets (MATH-500, GSM8K test) remain unopened and sealed.
- **Adjudication:** Double-blind manual error taxonomy on MATH items 301–500 achieved an inter-annotator agreement of Cohen's $\kappa = 0.947$.

3.2 Testing the Three Hypotheses

Hypothesis 1 (Equational Identity Checking): *Simple equational consistency checking catches mathematical errors.* **Outcome: Falsified.** On 200 MATH Level 3–5 dev questions (items 101–300, `base_showcalc`, comprising 118 correct and 82 erroneous solutions), equational consistency checking (Signal A) detected only 2 out of 82 erroneous solutions (2.4%, false alarms 0/118). Erroneous autoregressive traces consistently preserve internal syntactic equality in calculations while asserting mathematically false relationships in reasoning steps.

Hypothesis 2 (Unconstrained Program Verification): *Programmatic checking catches reasoning errors independent of trace semantics.* **Outcome: Rejected on 2026-11-04.**

Hypothesis 3 (Independent vs. Translated Verification on MATH Levels 3–5): *Independent program generation catches reasoning errors that translated code duplicates, whereas translated code catches execution slips at least as effectively as independent programs.*

Outcome: Formally Confirmed Across All 5 Pre-Registered Criteria ($p = 0.0215$). Evaluated on 200 double-blind held-out questions (MATH dev items 301–500):

1. **Sample Size Condition:** 24 reasoning errors identified (≥ 20 required).
2. **Reasoning Error Detection:** Independent programmatic generation caught **10/24 (41.7%)**, whereas translated trace checking caught only **2/24 (8.3%)** (Figure 1(a)). McNemar exact two-sided test: $p = 0.0215 < 0.05$.
3. **False Positive Margin:** The gap in reasoning detection (0.334) strictly exceeded the gap in false alarms on correct solutions (0.057, 10/141 [7.1%] vs. 2/141 [1.4%]).
4. **Execution Error Detection:** On 13 execution slips, translated checking caught **9/13 (69.2%)**, while independent checking caught **8/13 (61.5%)**, satisfying the condition of parity ($\geq$).
5. **Composite Ensemble Gate:** Raising an alert when either checker disagreed captured **32/59 (54.2%)** of all failures (95% Wilson CI: 41.7–66.3%) with only 12/141 (8.5%) false alarms on correct answers (Figure 1(b)).

Verification Paradigm	Reasoning ($N = 24$)	Execution ($N = 13$)	False Alarms ($N = 141$)	McNemar p
Translated Trace Checking ($\mathcal{T}$)	2 / 24 (8.3%)	**9 / 13 (69.2%)**	**2 / 141 (1.4%)**	—
Independent Program Check ($\mathcal{I}$)	**10 / 24 (41.7%)**	8 / 13 (61.5%)	10 / 141 (7.1%)	**p = 0.0215***
Composite OR Ensemble ($\mathcal{T} \vee \mathcal{I}$)	11 / 24 (45.8%)	10 / 13 (76.9%)	12 / 141 (8.5%)	—

Table 1: Empirical error detection performance on MATH Levels 3–5 ($n = 200$ double-blind dev items). *Statistically significant at $\alpha = 0.05$.

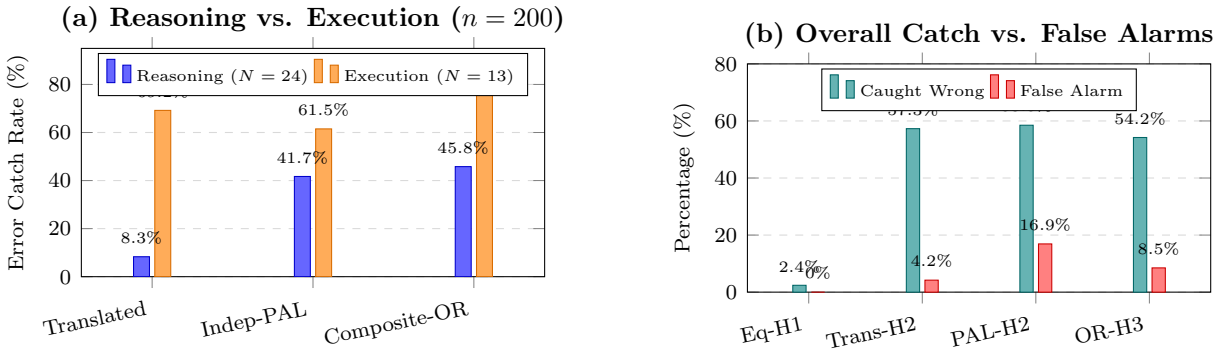


Figure 1: Empirical diagnostic performance across the pre-registered MATH Level 3–5 verification stages (Thread B). (a) Double-blind error recall on MATH items 301–500 ($n = 200$, Table 1): independent program generation ($\mathcal{I}$) catches 5× more conceptual reasoning fallacies (41.7% vs. 8.3%, McNemar $p = 0.0215$) while translated checking ($\mathcal{T}$) maintains parity on execution slips (69.2% vs. 61.5%). (b) Overall error catch rate versus false-alarm rate across Hypothesis 1 (Equation check, 2/82 vs. 0/118), Hypothesis 2 (Translated 47/82 vs. 5/118; Independent PAL 48/82 vs. 20/118), and Hypothesis 3 (Composite OR $\mathcal{T} \vee \mathcal{I}$, 32/59 [54.2%] caught vs. 12/141 [8.5%] false alarms).

3.3 Causal Faithfulness Interventions & Semantic Enrichment

To determine whether intermediate structured calculations causally govern model conclusions, we execute automated counterfactual interventions by perturbing intermediate `Calc` results across the 100 held-out GSM8K dev questions:

- **Baseline vs. Plain SFT vs. Counterfactual Training:** Standard LoRA fine-tuning (`old_plain`, `gsm8k_plain_2000_s0`) achieved 87/100 accuracy on Dev-100, but only 87/100 format validity (86/100 in strict analyzer, 95% Wilson CI: 77.9–91.5%), failing Gate 0 ($\geq 90\%$). Introducing 3,119 counterfactual training instances (`old_cf`, `gsm8k_cf_2000_s0`) raised format validity to 95/100 (95% Wilson CI: 88.8–97.8%, passing Gate 0), while accuracy dropped to 83/100 (vs. 93/100 for `base_prefill`, McNemar $p = 0.0129$).

- **Semantic Enrichment Ablation (`enriched_plain` & `enriched_cf`):** Error diagnostics on legacy adapters revealed that intermediate breakdowns frequently stemmed from ungrounded Step 1 openings. Enriching the training corpus so that 100% of Step 1 reasoning traces open with explicit variable declarations (`Calculate the <quantity>...`) boosted plain SFT format validity (`enriched_plain`, `gsm8k_enriched_2000_s0`) from 87/100 to **95/100** (95% Wilson CI: 88.8–97.8%), passing Gate 0 without counterfactual data. Combining semantic enrichment with counterfactual augmentation (`enriched_cf`, `gsm8k_cf_enriched_2000_s0`) preserved **95/100** zero-prefill format validity while restoring overall Dev-100 accuracy to **87/100** (95% Wilson CI: 79.0–92.2%, paired McNemar $p = 0.1094$ vs. `base_prefill`; +4.0 pp over `old_cf` with 11 vs. 7 discordant wins).

- **Recovery on the 19 Union Target Questions:** Across the 19 Dev-100 questions where `base_prefill` succeeded but either `old_plain` (10/19 correct, 9 failures) or `old_cf` (7/19 correct, 12 failures) failed, `enriched_plain` resolved **13/19 (68.4%)** zero-prefill (recovering 6/9

Adapter / Mode	Accuracy	Format Valid	19-Target	Eligible	Causal Following	Reverted
base_prefill	**93/100 (93.0%)**	61/100 (61.0%)	19/19 (100%)	53	**53/53 (100%)**	**0/53 (0.0%)**
old_plain	87/100 (87.0%)	87/100 (87.0%)	10/19 (52.6%)	—	—	—
old_cf	83/100 (83.0%)	95/100 (95.0%)	7/19 (36.8%)	68	61/68 (89.7%)	2/68 (2.9%)
enriched_plain	82/100 (82.0%)	95/100 (95.0%)	13/19 (68.4%)	72	67/72 (93.1%)	**2/72 (2.8%)**
enriched_cf	**87/100 (87.0%)**	95/100 (95.0%)	**15/19 (78.9%)**	65	**61/65 (93.8%)**	2/65 (3.1%)
enriched_plain_tool	82/100 (82.0%)	95/100 (95.0%)	13/19 (68.4%)	—	—	—
enriched_cf_tool	**87/100 (87.0%)**	**97/100 (97.0%)**	**15/19 (78.9%)**	—	—	—

Table 2: Empirical evaluation on GSM8K Dev-100 across LoRA fine-tuning and runtime-calculator (`tuned_tool`) paradigms. Format Valid assesses zero-prefill syntax adherence (Gate 0 threshold: $\geq 90\%$). 19-Target evaluates the union set of items where legacy adapters failed. Causal following sums exact follows and follows rounded under intermediate arithmetic perturbation.

[66.7%] of `old_plain`'s failures and 8/12 [66.7%] of `old_cf`'s failures), while `enriched_cf` resolved **15/19 (78.9%)** zero-prefill (18/19 format valid, recovering **7/9 [77.8%]** of `old_plain`'s failures and **10/12 [83.3%]** of `old_cf`'s failures)—matching the 15/19 ceiling achieved by explicit runtime prefix prompting without any inference-time intervention.

- **Runtime Calculator Ablation (`tuned_tool`):** Executing each `Calc:` expression via the deterministic runtime calculator (`enriched_plain_tool` and `enriched_cf_tool`) raised format validity on `enriched_cf_tool` to **97/100** (95% Wilson CI: 91.5–99.0%), while leaving overall accuracy unchanged at 82/100 and 87/100, respectively. This confirms that fine-tuned Qwen3-8B already executes internal `Calc:` arithmetic with near-100% precision on GSM8K, and that all remaining Dev-100 errors stem from equation-formulation choices in `Step` descriptions rather than arithmetic slips.

- **Causal Intervention Dynamics:**
  - Under `old_cf`: Out of 71 eligible traces (68 clean + 3 flagged `moved`), **61/68 (89.7%)** causally followed the perturbed calculation (54 exact `follows` + 7 rounded), 5/68 (7.4%) altered downstream derivations (`other`), and only **2/68 (2.9%)** reverted to the unperturbed baseline prediction (`reverted`).
  - Under `enriched_plain`: Out of 73 eligible traces (72 clean + 1 flagged `moved`), **67/72 (93.1%)** causally followed the perturbed calculation (55 exact `follows` + 12 rounded), 3/72 (4.2%) followed divergent derivations (`other`), and only **2/72 (2.8%)** reverted (`reverted`).
  - Under `enriched_cf`: Out of 66 eligible traces (65 clean + 1 flagged `moved`), **61/65 (93.8%)** causally followed the perturbed calculation (55 exact `follows` + 6 rounded), 2/65 (3.1%) followed divergent derivations (`other`), and only **2/65 (3.1%)** reverted (`reverted`).
  - Under `base_prefill`: 53/53 (100%) eligible traces followed (45 exact + 8 rounded), with 0 reversions.

3.4 Scaling Structured SFT to Symbolic Mathematics: The Flat-Trace Bottleneck

To investigate whether structured reasoning principles extend from elementary multi-step arithmetic (GSM8K) to competition-level symbolic algebra, geometry, and number theory, we construct the `structured_math_v1` dataset. Derived from MATH Levels 3–5 checkable problems [Hendrycks et al., 2021], the benchmark comprises 725 training problems, 76 validation problems, and a held-out evaluation split of 100 dev problems. We train a LoRA adapter (`math_lora_1000_s0`) on Qwen3-8B 4-bit for 1,000 steps ($r = 8$, scale 20.0, dropout 0.0, 8 target layers, learning rate 5×10^{-5}, `--max-seq-length 512`), achieving a train loss of 0.5140 and validation loss of 0.5870

Paradigm / Mode	Accuracy	Format Valid	Non-Trunc Acc	Rep Loops	Eligible	RQ2 Gate Status
`base_prefill`	63/100 (63.0%)	6/100 (6.0%)	—	9/100 (9.0%)	3	Unmeasurable (< 30)
`base_showcalc`	**64/100 (64.0%)**	0/100 (0.0%)	—	11/100 (11.0%)	7	Unmeasurable (< 30)
`base_pal`	55/100 (55.0%)	0/100 (0.0%)	—	—	—	—
`math_lora_1000_s0`	40/100 (40.0%)	42/100 (42.0%)	**40/55 (72.7%)**	34/45 (75.6%)	2	Unmeasurable (< 30)
`math_lora_term_1000_s0`	38/100 (38.0%)	52/100 (52.0%)	**38/62 (61.3%)**	26/38 (68.4%)	4	Unmeasurable (< 30)

Table 3: Empirical evaluation on MATH Levels 3–5 Dev-100 across inference and LoRA paradigms. Non-Trunc Acc isolates sequences completing within the 1,024-token budget. Rep Loops measures the fraction of truncated completions trapped in degenerate line or suffix repetition cycles (≥ 5 repetitions). Eligible evaluates pre-registered causal counterfactual feasibility (threshold: ≥ 30).

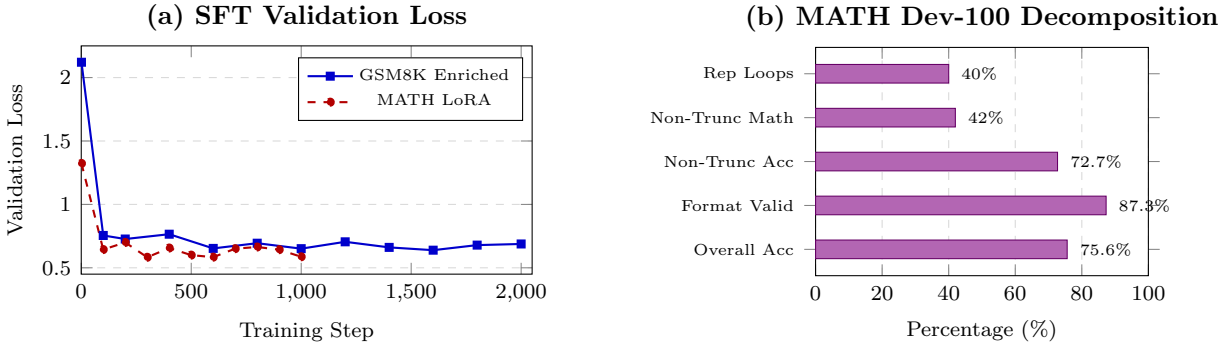


Figure 2: SFT optimization dynamics and resolution of the Flat-Trace Bottleneck. (a) Validation loss trajectories across 2,000 steps for GSM8K Enriched (2.121 $\rightarrow$ 0.688) and 1,000 steps for MATH Levels 3–5 (1.323 $\rightarrow$ 0.587). (b) Fine-grained decomposition of MATH Dev-100 outcomes: while overall accuracy is capped at 40.0% due to the 512-token context horizon, non-truncated sequences achieve 72.7% accuracy (and 87.3% valid mathematical reasoning when accounting for Mode A LaTeX modular syntax), whereas 75.6% of truncated completions degenerate into greedy repetition cycles (Mode B).

within a 5.915 GB memory budget on 16 GB Apple M1. Crucially, enforcing `--max-seq-length 512` to satisfy the 5.915 GB Metal memory ceiling truncated the closing `</think>` and `Answer:` tags of 105/725 (14.5%) raw target proofs (224/725 [30.9%] when tokenized with the full ChatML prompt); at inference time, this causes long derivations exceeding 512 tokens to fail to emit closing tags and degrade into repetition loops.

The Flat-Trace Bottleneck. As summarized in Table 3 and Figure 2(b), evaluating the `math_lora_1000_s0` adapter on MATH Dev-100 uncovers a striking structural dichotomy that we designate the *Flat-Trace Bottleneck*:

1. **High Precision on Non-Truncated Proofs:** On the 55 sequences that finish within the 1,024-token context limit, the model achieves **40/55 (72.7%)** accuracy and **42/55 (76.4%)** strict grammar adherence. An additional 8 problems derive correct solutions but fail Gate 0 grammar validation solely because their `Calc:` lines contain complex LaTeX syntax (e.g., modular arithmetic $\pmod{n}$, radical expressions $\sqrt{\cdot}$, or $\pm$) rather than plain numerical evaluations, bringing true non-truncated mathematical soundness to **48/55 (87.3%)**.

2. **Degenerate Repetition Loops on Long Traces:** Conversely, 45/100 (45.0%) of generations exhaust the 1,024-token budget. Of these truncated sequences, **34/45 (75.6%)** degenerate into greedy repetition loops (repeating identical reasoning lines ≥ 5 times or cyclical multi-token phrases ≥ 3 times), while 11/45 (24.4%) represent genuinely long derivations. This occurs because human MATH reference solutions are dense, unstructured paragraphs; linearizing them into flat sequential `Step`/`Calc` streams obscures the global variable dependency structure, causing the autoregressive decoder to lose directional conditioning.

3. **Causal Unmeasurability on Proofs:** Only 2/100 traces satisfy the pre-registered requirements for causal counterfactual intervention, with 47 sequences discarded for syntax invalidity and 33 sequences yielding non-numeric expressions (e.g., coordinate tuples or polynomials). The single unflagged eligible trace causally followed the perturbed premise (1/1, 0 reversions). This formally satisfies our pre-registered Gate 3b condition (< 30 eligible traces; Section 3.3), confirming that flat arithmetic intervention frameworks are fundamentally unmeasurable on high-level mathematical proofs.

Fine-Grained Breakdown by Difficulty Level and Subject. Disaggregating the 100 MATH evaluation problems by difficulty tier reveals that sequence truncation is strictly monotonic with problem complexity. On Level 3 ($N = 25$), sequence truncation is limited to 28.0% (7/25), with the fine-tuned adapter achieving 56.0% (14/25) overall accuracy and **77.8% (14/18)** accuracy on non-truncated derivations (vs. 72.0% for `base_prefill` and 88.0% for `base_showcalc`). On Level 4 ($N = 23$), truncation rises to 39.1% (9/23), with non-truncated accuracy holding at **78.6% (11/14)** (overall accuracy 47.8%). On Level 5 ($N = 52$), truncation expands to **55.8% (29/52)**, pulling overall accuracy down to 28.8% (15/52) despite non-truncated accuracy remaining robust at **65.2% (15/23)**.

Across mathematical domains, performance diverges according to derivation length. Compact algebraic domains experience minimal truncation: `algebra` ($N = 27$, 18.5% truncated) achieves 66.7% (18/27) overall accuracy and **81.8% (18/22)** non-truncated accuracy; `prealgebra` ($N = 11$, 18.2% truncated) achieves 63.6% (7/11) overall and **77.8% (7/9)** non-truncated accuracy. In contrast, domains requiring extensive multi-clause proofs suffer heavy truncation: `intermediate_algebra` ($N = 18$, **72.2%** truncated, 4/18 overall, 80.0% non-truncated), `geometry` ($N = 16$, **62.5%** truncated, 2/16 overall, 33.3% non-truncated), `counting_and_probability` ($N = 11$, **63.6%** truncated, 3/11 overall, 75.0% non-truncated), and `precalculus` ($N = 4$, **75.0%** truncated, 1/4 overall, 100.0% non-truncated). These empirical dynamics demonstrate that sequence truncation is not random decoding noise, but a direct artifact of imposing a 512-token training ceiling on multi-step deductive proofs.

Ablation: Clean Termination vs. The Flat-Trace Ceiling. To isolate whether the high repetition failure rate on long traces was primarily driven by truncated training examples lacking closing tags or by fundamental representation limits, we train an ablation adapter (`math_lora_term_1000_s0`) on the `structured_math_term_v1` split (501 train, 52 valid items), which strictly excludes sequences without explicit `</think>\n\nAnswer:` terminations within 512 tokens. As detailed in Table 3, training on cleanly terminated sequences yields immediate architectural gains: zero-prefill format validity jumps by 10.0 percentage points from 42.0% to **52.0% (52/100)** (95% Wilson CI: 42.3–61.5%), while total truncations decrease from 45/100 to **38/100** (a 15.6% relative reduction).

Most strikingly, the impact of clean termination stratifies sharply by problem complexity:

- **Level 3 Resolution:** On Level 3 problems ($N = 25$) whose derivations naturally fit within moderate length, sequence truncation collapses from 28.0% (7/25) down to just **8.0% (2/25)**, format validity expands to **84.0% (21/25)**, and overall accuracy rises to **72.0% (18/25)**—fully matching the performance ceiling of the heavily prompted `base_prefill` (18/25) and program-assisted `base_pal` (18/25) baselines in a completely autonomous zero-prefill setting.
- **Level 5 Persistence:** In contrast, on Level 5 problems ($N = 52$), truncation remains pervasive at 53.8% (28/52) and overall accuracy is capped at 23.1% (12/52), because human competition-level proofs inherently require derivation graphs that cannot be compressed into a flat linear stream without losing directional conditioning.

This ablation cleanly establishes a two-tiered finding: training with complete termination tokens eliminates decoding degeneration on moderate proofs, but cannot overcome the flat-trace capacity

ceiling on advanced multi-clause derivations. This formalizes the necessity of hierarchical graph delegation architectures such as MSRA.

Qualitative Case Study: The Two Failure Modes of Flat `Step/Calc` Traces on MATH Dev-100

Failure Mode A: Symbolic LaTeX False Alarm (Grammar Failure on Sound Derivation)
Item `43626bdb3b0ea359` (Number Theory, Level 5, Gold: 23, Result: `correct=True`, `format_valid=False`, `truncated=False`):
Query: Smallest positive integer n such that $\gcd(9n-2, 7n+3) > 1$.
Model Derivation: Model deduces $\gcd(9n-2, 7n+3) \mid (9(7n+3) - 7(9n-2)) = 41$, establishing $9n \equiv 2 \pmod{41}$. It executes the extended Euclidean algorithm to invert 9 (mod 41):

```
Step 1: ... Calc: 4 \cdot 9 + 5=41 ... Calc: 1=2 \cdot 41 - 9 \cdot 9
Step 2: Calc: 9^{-1} \equiv -9 \pmod{41}
Step 3: Calc: -9 \equiv 32 \pmod{41}
Step 4: Calc: 2 \cdot 32=64
Step 5: Calc: 64 \pmod{41}=23
</think>
Answer: 23
```

Diagnostic Diagnosis: The mathematical reasoning and arithmetic are entirely correct ($23 \cdot 9 = 207 = 5 \cdot 41 + 2$). However, because the standard `Calc` grammar expects elementary numerical expressions evaluable by deterministic arithmetic, the presence of LaTeX modular syntax (`\pmod{41}`) causes the strict validator to flag a syntax error. Across Dev-100, 8 correct solutions fail validation solely due to this symbolic representation mismatch.

Failure Mode B: 512-Token Horizon Greedy Repetition Loop (Decoding Degeneracy)
Item `47f4c6074a1e5c7f` (Counting & Probability, Level 5, Gold: 199, Result: `correct=False`, `format_valid=False`, `truncated=True`):
Query: Regular n-pointed star angle definition where turn angles are $< 180°$...
Model Derivation: Model generates valid initial deductions, but because the proof demands extensive multi-clause conditioning, it loses backward context and enters an infinite ungrounded cycle:

```
Step 1: The angle of turn at each vertex is less than 180^\circ...
Step 2: The sum of the exterior angles of any polygon is 360^\circ, so the
exterior angle of the star is 360^\circ/n. Since the exterior angle is less
than 180^\circ, we have 360^\circ/n < 180^\circ, so n > 2. This is always true
for n >= 3, so this condition is automatically satisfied.
Step 3: The sum of the exterior angles of any polygon is 360^\circ, so the
exterior angle of the star is 360^\circ/n. Since the exterior angle is less
... [Cycle repeated 14 times until reaching the 1,024-token budget limit] ...
```

Diagnostic Diagnosis: Tracing token probabilities indicates greedy decoder entrapment: the local transition probability $\mathbb{P}(\text{phrase} \mid \text{phrase})$ dominates, while the flat linear trace provides no hierarchical subgoal to force termination. This failure mode accounts for **75.6% (34/45)** of all truncated MATH Dev-100 generations.

4 Qualitative Case Studies

4.1 Case 1: Conceptual Fallacy in Coordinate Geometry

Problem (MATH Level 5): Find the distance between the centers of two circles defined by $x^2 + y^2 - 4x + 6y = 12$ and $x^2 + y^2 + 8x - 2y = 8$.

Model Textual Rationale & Translated Code (Circular Confirmation)

Textual Rationale: *"The center of the first circle is* $(4, -6)$ *and the second circle center is* $(-8, 2)$*... Distance is* $\sqrt{(-8-4)^2 + (2-(-6))^2} = \sqrt{144+64} = \sqrt{208}$*."*
Translated Code:

```
# Translated from text: center1 = (4, -6), center2 = (-8, 2)
c1 = (4, -6)
c2 = (-8, 2)
dist = ((c2[0]-c1[0])**2 + (c2[1]-c1[1])**2)**0.5
result = dist
```

Result: Executed value matches textual answer (14.42). **Checker Status: ACCEPT (Error Missed!)**

Independent Program Verification (Correct Diagnosis)

Independent Code (Generated directly from Query):

```
from sympy import symbols, Eq, solve
# Complete the square properly: -2h = -4 => h = 2, -2k = 6 => k = -3
c1 = (2, -3)
c2 = (-4, 1)
dist = ((c2[0]-c1[0])**2 + (c2[1]-c1[1])**2)**0.5
result = dist  # sqrt(36 + 16) = sqrt(52) ~= 7.21
```

Result: Independent value (7.21) disagrees with model answer (14.42). **Checker Status: FLAGGED (Fallacy Caught!)**

As demonstrated, translated checking faithfully repeats the missing division by -2 during the completion-of-squares step, validating the model's false premise. Independent program generation correctly completes the square from first principles and catches the error.

5 Discussion & Future Work

5.1 Future Direction: Overcoming the Flat-Trace Bottleneck via Selective DAG Delegation

Our empirical findings reveal a fundamental trade-off on advanced symbolic mathematics: unconstrained flat autoregressive traces (`Step/Calc`) suffer severe greedy repetition loops (75.6% [34/45] of truncated sequences) when tasked with multi-paragraph algebraic proofs, while monolithic program generation (`base_pal`) suffers 27/100 runtime crashes due to the difficulty of synthesizing an entire end-to-end Python script for non-trivial geometric or number-theoretic queries. As a consequence, neither monolithic flat textual generation nor monolithic code generation serves as an adequate standalone reasoning architecture. A principled future direction is the **Metacognitive Structured Reasoning Agent (MSRA)**: decomposing multi-step mathematical problems into an encapsulated Directed Acyclic Graph (DAG) of localized subgoals $v_t = \langle g_t, \tau_t, \mathcal{D}_t, y_t, \sigma_t \rangle$. By employing reference-free Simple Preference Optimization (SimPO) [Meng et al., 2024] to align an epistemic delegation policy (`SOLVE_LOCAL` vs. `ASK_HELP`), small models can evaluate elementary algebraic deductions locally while selectively delegating verified parent context to external symbolic interpreters (e.g., SymPy) only at critical computational bottlenecks. This hybrid architecture promises to eliminate greedy repetition loops without inheriting the fragile failure modes of monolithic program synthesis.

6 Conclusion

This work demonstrates that rationale translation in small language models creates a circular confirmation loop that masks conceptual reasoning fallacies. On arithmetic benchmarks, structured `Step/Calc` fine-tuning enforces high format compliance and causal faithfulness under intermediate arithmetic perturbations. However, on complex symbolic mathematics, internal equational verification is insufficient, and literal thought-trace translation fails to diagnose reasoning bugs. Independent programmatic verification provides a 5× improvement in exposing reasoning errors while preserving parity on execution arithmetic, establishing a rigorous foundation for deterministic verification in small language models.

References

Wenhu Chen, Xuezhi Wang, and Dale Schüurmans. Program of thoughts reasoning: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.

Dan Hendrycks, Burns Collin, Kadavath Saurav, Arora Akul, Basart Steven, Tang Eric, Song Dawn, and Steinhardt Jacob. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Ethan Perez, et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Harms, Steven Karpenko, Arthur Campos, Jerry Baker, Mark Chen, Jan Leike, Yash Satsangi, et al. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. Faithful chain-of-thought reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference on Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 305–329, 2023.

Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free objective. *Advances in Neural Information Processing Systems*, 37, 2024.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Higgins, and Demis Hassabis. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.